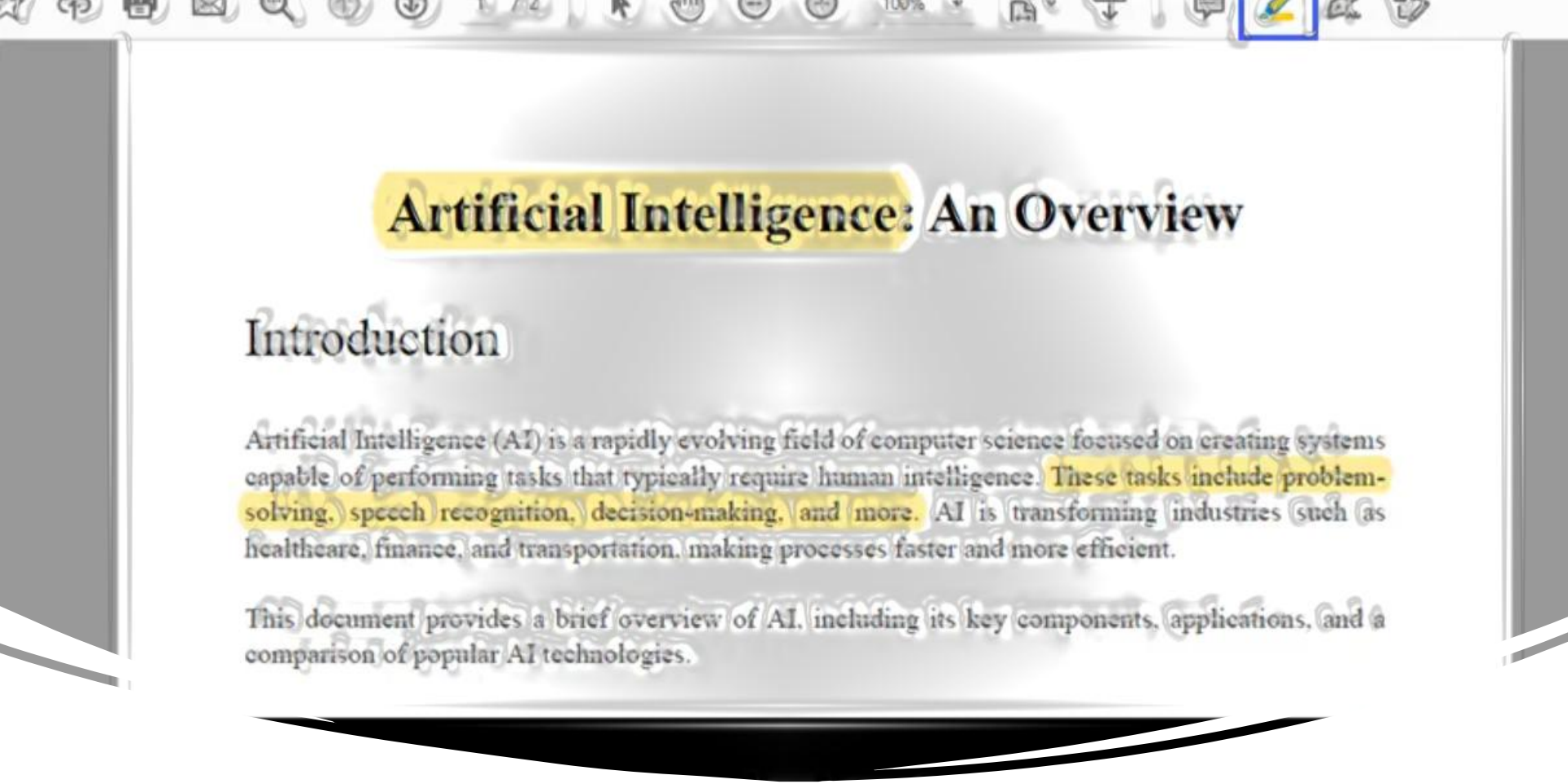
Mouse Interaction in HCI

Dr. Salwa Osama



Artificial Intelligence: An Overview

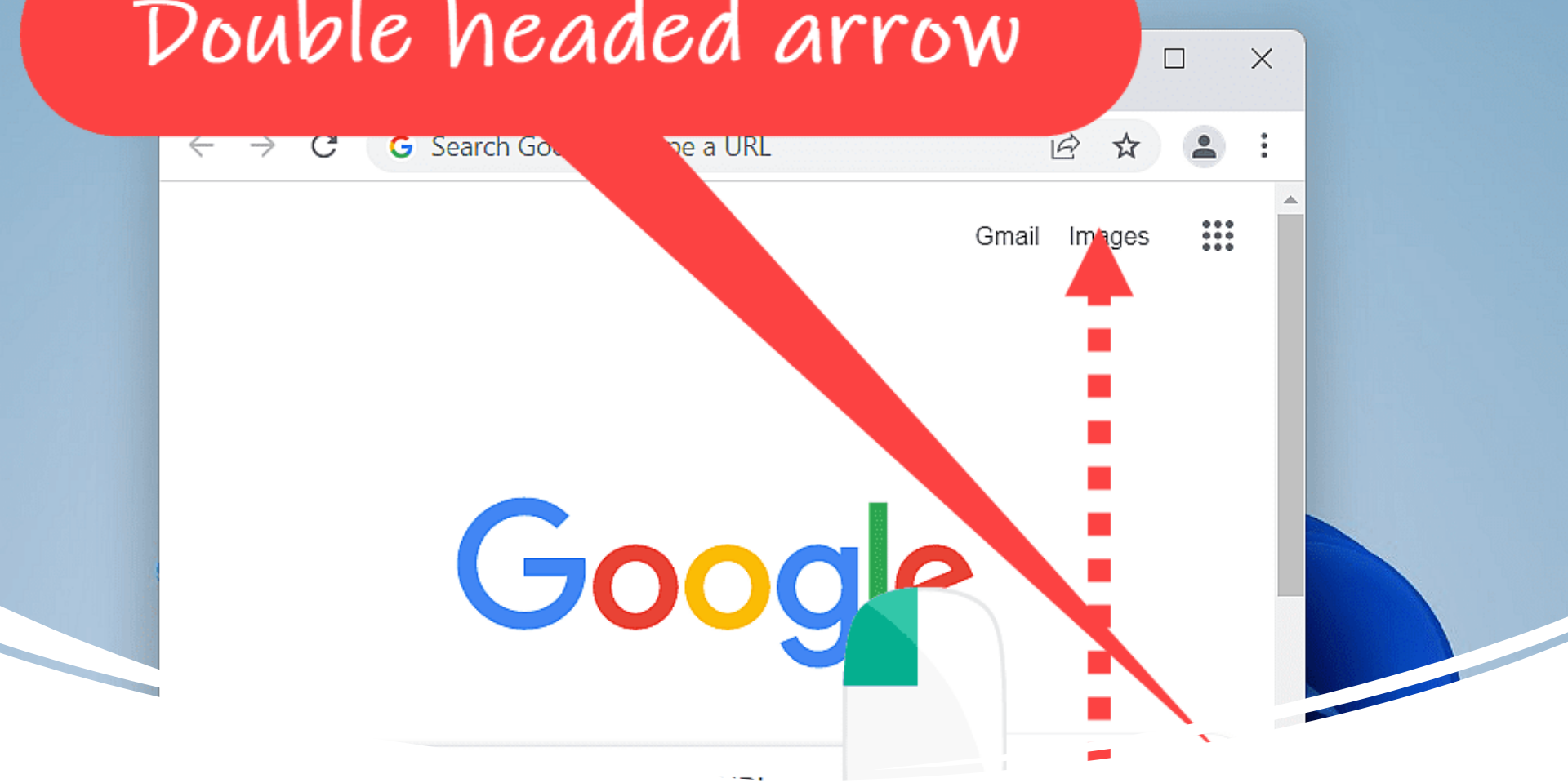
Introduction

Artificial Intelligence (AI) is a rapidly evolving field of computer science focused on creating systems capable of performing tasks that typically require human intelligence. These tasks include problem-solving, speech recognition, decision-making, and more. AI is transforming industries such as healthcare, finance, and transportation, making processes faster and more efficient.

This document provides a brief overview of AI, including its key components, applications, and a comparison of popular AI technologies.

Motivation

- Drawing with the mouse is not just a game; it is a fundamental example of human–computer interaction. Similar interactions happen when you:
  - Highlight text with a mouse drag.



Motivation

- Drawing with the mouse is not just a game; it is a fundamental example of human–computer interaction. Similar interactions happen when you:
  - Resize a window by dragging its corner.

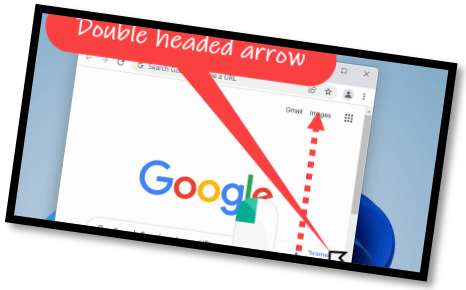
Motivation

- Drawing with the mouse is not just a game; it is a fundamental example of human–computer interaction. Similar interactions happen when you:
  - Play digital drawing or design tools (like Photoshop or Paint).

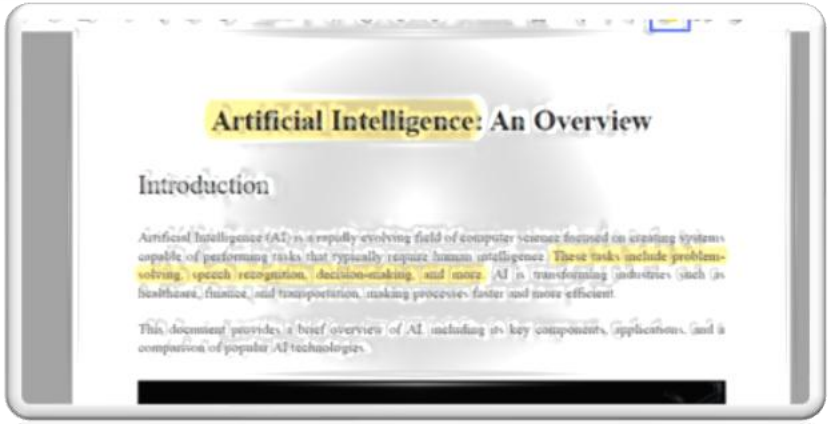
Motivation

- Drawing with the mouse is not just a game; it is a fundamental example of human–computer interaction. Similar interactions happen when you:
  - Interact with touchscreens when sketching or writing with a finger or stylus

Motivation



All of these show how simple actions (dragging, clicking, touching) are translated into meaningful computer operations.



Paper Application (1) - Mouse Gesture Recognition for Human Computer Interaction

- In Computer Science today, focus has moved from **system-oriented software** (just working correctly) to **user-oriented software** (easy and enjoyable to use).
- User Experience (UX) is now a key part of software design.
- This paper presents a way to interact with the computer using **mouse gestures** (drawing shapes with the mouse).
- The goal: **make interaction faster, simpler, and more accessible**, especially for people with disabilities.
- The algorithm is based on a **Hidden Markov Model (HMM)**, which is a statistical method for recognizing gesture patterns.

Gesture	Description of Gesture Path	Gesture	Description of Gesture Path
	Down - Right		Down - Left
	Clock-wise Rectangle*		Counter Clock-wise Rectangle*
	Folder Shape*		Right - Left#
	Down - Up#		Right - Left - Right - Down - Left - Up*#
	Left - Down - Right - Down - Left (squarish S)		Right - Down - Left - Up - Right
	Left - Down - Right - Up - Left		

, K. M. M. P. H. S. D. S. B. P. (2015). Mouse Gesture Recognition for Human Computer Interaction. *International Journal on Recent and Innovation Trends in Computing and Communication*, *3*(5), 2517–2519. https://doi.org/10.17762/ijritcc.v3i5.4275

Paper Application (1) -

Mouse Gesture Recognition for Human Computer Interaction

- **User Experience (UX)** means how a person **feels and interacts** when using a system, software, website, or app.
- It is not only about whether the software *works*, but also:
  - Is it easy to use or too complicated?
  - Does it help the user finish tasks quickly?
  - Does the user feel comfortable or frustrated while using it?
- Hidden Markov Mode is used for mouse gesture recognition

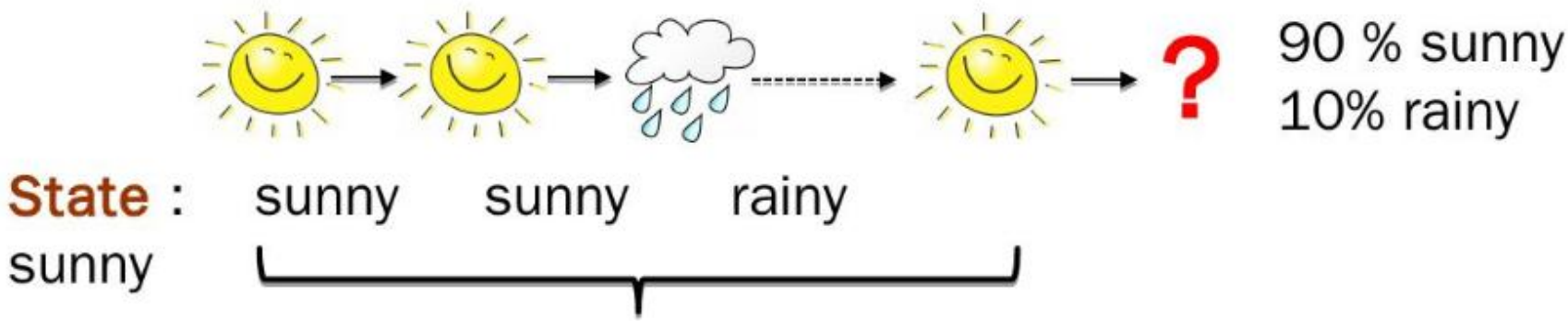
, K. M. M. P. H. S. D. S. B. P. (2015). Mouse Gesture Recognition for Human Computer Interaction. *International Journal on Recent and Innovation Trends in Computing and Communication*, *3*(5), 2517–2519. https://doi.org/10.17762/ijritcc.v3i5.4275

Paper Application (1) -

Mouse Gesture Recognition for Human Computer Interaction

Hidden Markov Model (HMM):

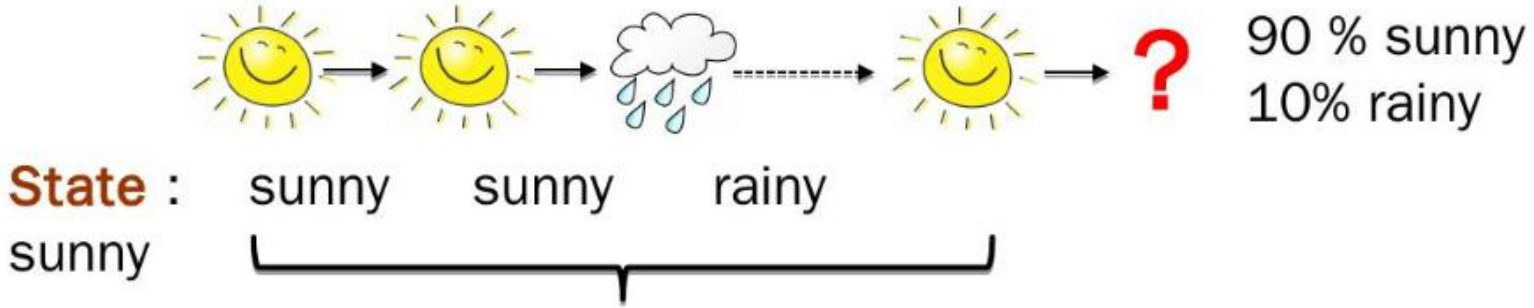
- A mathematical model that helps recognize **sequences** by learning how one step leads to another.
- If today is sunny, tomorrow is probably sunny.
- If today is rainy, tomorrow is likely rainy.
- We don’t know for sure, but we use probabilities.
- Observed events = mouse positions while drawing.
- Hidden states = the gesture being made (circle, line, etc.).



Paper Application (1) -

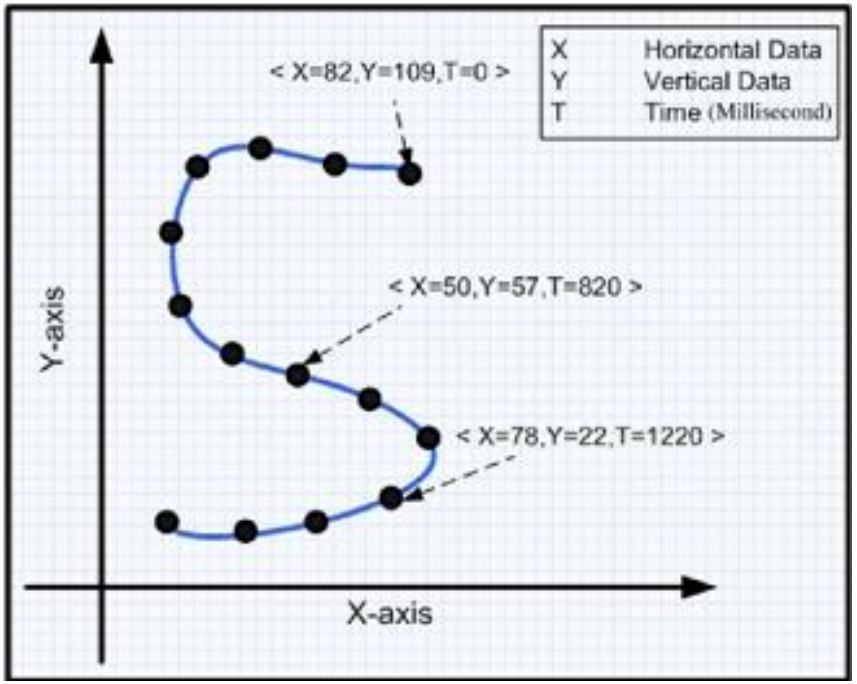
Mouse Gesture Recognition for Human Computer Interaction

- **Why Use HMM in Mouse Gesture Recognition?**
- **Gestures are sequences:**
  Drawing = many small movements (like speech = many sounds).
- **Handles variation:**
  Works even if user draws **fast/slow, big/small, messy/neat**.
- **Probabilistic approach:**
  Accepts **imperfect gestures** and still guesses the most likely one.
- Circle gesture sequence = (right → down → left → up → right).
- HMM learns this pattern.
- Later, when drawn again, HMM recognizes it as a circle even if it's not perfect.



Paper Application (2) - Biometric Authentication Using Mouse Gesture Dynamics

- **What is Mouse Dynamics?**
  - Mouse dynamics = how people move and use the mouse.
  - Every person has a **unique style** of moving the mouse.
  - It can be used as a **biometric** (like fingerprint or face).
- **Example:**
  - Some people move the mouse fast, others slow.
  - Some make smooth curves, others sharp moves.
- The authors built a system that uses mouse gestures (drawn shapes) to check a person's identity.
  They collect the gestures, study how they are drawn, and then use a Learning Vector Quantization (LVQ) Neural Network to recognize the user.



B. Sayed, I. Traoré, I. Woungang and M. S. Obaidat, "Biometric Authentication Using Mouse Gesture Dynamics," in *IEEE Systems Journal*, vol. 7, no. 2, pp. 262-274, June 2013, doi: 10.1109/JSYST.2012.2221932. keywords: {Computer security;Computer peripherals;Biometrics (access control);Pattern classification;Graphical user interfaces;Behavioral biometrics;biometric authentication;computer security;identity verification;mouse dynamics},

Simple Example 1

Defines a **callback function** named draw_circle that will be called automatically whenever a mouse event happens inside the window.
event: tells what kind of mouse action occurred (click, move, etc.).
x, y: the mouse position in the window.
flags and param: optional parameters (not used here).

Checks if the user **pressed the left mouse button**.
If true, the next lines will run.

```
cv2
img = cv2.imread("AI COdes/HCI Course/Basics of HCI/eye.jpg")
def draw_circle(event, x, y, flags, param):
    if event == cv2.EVENT_LBUTTONDOWN:
        print("hello")
        cv2.circle(img, (x, y), 100, (0, 255, 0), -1)

cv2.namedWindow(winname = "Title of Popup Window")
cv2.setMouseCallback("Title of Popup Window", draw_circle)
while True:
    cv2.imshow("Title of Popup Window", img)

    if cv2.waitKey(10) & 0xFF == 27:
```

Draws a **filled green circle** on the image.
(x, y) → the center of the circle (where the user clicked).
100 → radius of the circle in pixels.
(0, 255, 0) → color in **BGR format** (green).
-1 → means **fill the circle** instead of just drawing the outline.

Simple E[illegible] 1

Creates a new window and gives it the name **"Title of Popup Window"**.
This name will be used to show the image and attach the mouse callback.

Links the **mouse event function** (draw_circle) to the created window.
Now, any mouse action inside this window will trigger that function.

```
cv2
cv2.imread("AI COdes/HCI Course/[illegible] HCI/eye.jpg")
aw_circle(event, x, y, flags [illegible]):
 event == cv2.EVENT_LBUTT[illegible]N:
  print("hello")
  cv2.circle(img [illegible], y), 100, (0, 255, 0), -1)

cv2.namedWindow(winname = "Title of Popup Window")
cv2.setMouseCallback("Title of Popup Window", draw_circle)
while True:
    cv2.imshow("Title of Popup Window", img)

    [illegible]
```

Displays the image in a continuous **loop**.
Every time you click, the image is updated with the new circle.

ple Example 2

ix and iy store the starting coordinates of the rectangle. drawing is a flag to check if the mouse is currently being pressed and dragged.

```
ix = -1
iy = -1
drawing = False
def draw_rectangle_with_drag(event,

    global ix, iy, drawing, img

    if event == cv2.EVENT_LBUTTONDOWN:
        drawing = True
        ix = x
        iy = y

    elif event == cv2.EVENT_MOUSEMOVE:
        if drawing == True:
            cv2.rectangle(img, pt1 =(ix, iy),
                          pt2 =(x, y),
                          color =(0, 255, 255),
                          thickness =-1)
    elif event == cv2.EVENT_LBUTTONUP:
        drawing = False
        cv2.rectangle(img, pt1 =(ix, iy), pt2 =(x, y),color =(0, 255, 255),
```

Define a function that will be called automatically whenever a mouse event happens.The parameters give information about the event type and mouse position.

If the **left mouse button** is pressed: Set drawing = True (start drawing). Store the starting point (x, y) as (ix, iy).

Example 2

When the mouse moves (EVENT_MOUSEMOVE):
If drawing is True (the mouse button is still pressed),
draw a **filled yellow rectangle** (thickness = -1) from the start point (ix, iy) to the current point (x, y).

```
ix =
iy =
drawing =
def draw_rectang                    y, flags, par

    global ix, iy, drawing, img

    if event == cv2.EVENT_LBUTTONDOWN:
        drawing = True
        ix = x
        iy = y

    elif event == cv2.EVENT_MOUSEMOVE:
        if drawing == True:
            cv2.rectangle(img, pt1 =(ix, iy),
                          pt2 =(x, y),
                          color =(0, 255,  5),
                          thickness =-1)
    elif event == cv2.EVENT_LBUTTONUP:
        drawing = False
        cv2.rectangle(img, pt1 =(ix, iy), pt2 =(x, y),color =(0, 255, 255),
                      thickness =-1)
```

When the **left mouse button** is released:
Set drawing = False (stop drawing).
Draw the final rectangle on the image.

Painting Example
Kivy Library

- Kivy is an open-source Python library for building cross-platform GUI applications.
- It works on Windows, macOS, Linux, Android, iOS and Raspberry Pi.
- Kivy supports multi-touch gestures, mouse and keyboard inputs, making it ideal for modern interactive applications

pip install kivy

Painting Example

Each touch event in Kivy has a dictionary called ud (User Data). This line checks if a color has **already been assigned** to this touch. If not, we'll create one.

This function assigns a **unique and stable color** to each touch (finger or mouse click) on the screen.

```
re.window import Window
om

cla  ingPad(Widget):
    d  olor_for_touch(self, touch):
        # stable per-finger color
        if 'color' not in touch.ud:
            random.seed(touch.uid)
            touch.ud['color'] = (random.random(), random.random(), random.random())
        return touch.ud['color']

    def on_touch_down(self, touch):
        with self.canvas:
            Color(*self.color_for_touch(touch))
            d = 40  # circle diameter
            x, y = touch.pos
            # store things in touch.ud so we ca
            touch.ud['circle'] = Ellipse(pos=(x
            touch.ud['line'] = Line(points=[x, y
```

Every touch has a **unique ID** (uid) that identifies it. We use it to **initialize the random number generator** so that each finger gets a **consistent color** every time. This ensures that one finger always has the same color throughout its movement.

Painting Ex

This function is called automatically when you touch (press) the screen — either with your finger (on mobile) or with the mouse (on PC).

•Opens a **drawing space (canvas)** where you can draw shapes, colors, or lines.
•Anything inside this block will appear on the screen.

```
re.window import Window
m

ngPad(Widget):
lor_for_touch(self, touch):
stable per-finger color
'color' not in touch.ud:
random.seed(touch.uid)
touch.ud['color'] = (om.random(), random.random(), random.random())
eturn touch.ud['color']

def on_touch_down(self, touch):
    with self.canvas:
        Color(*self.color_for_touch(touch))
        d = 40  # circle diameter
        x, y = touch.pos
        # store things in touch.ud so we ca
        touch.ud['circle'] = Ellipse(pos=(x
        touch.ud['line'] = Line(points=[x, y
```

•Gets the **unique color** for this touch (from the previous function you asked about).
•The * unpacks the RGB values like (r, g, b).
•It sets this color as the **current drawing color**.

Painting E

Draws a **small circle** centered at the place you touched.
The position is adjusted (x - d/2, y - d/2) to make the circle centered on your finger, not offset.
The circle's info is saved in touch.ud (user data)

Gets the **position** of the touch on the screen

```
from kivy.app import App
                                                    lipse, Line
        y.core.window import Window
        andom

        awingPad(Widget):
        color_for_touch(self, touch):
        # stable per-finger color
        f 'color' not in touch.ud:
            random.seed(touch.uid)
            touch.ud['color'] = (rand                m()
        eturn touch.ud['color']

    de    _touch_down(self, tou
        th self.canvas:
            Color(*self.colo      or_touch(touch))
            d = 40  # cir      diameter
            x, y = touch.      pos
            # store th      ngs in touch.u      we c      extend the
            touch.ud['circle'] = Ellip      (pos=(x - d/2, y - d/2), size=(d
            touch.ud['line'] = Line(points=[x, y], width=2)
```

starts a **new line** from the current touch point.
The points list contains the coordinates where the line begins.
The width=2 makes the line slightly thicker.
We save this line in touch.ud so that when the finger moves, we can keep adding points to it (in on_touch_move).

g Example

Every time your finger moves, this function adds a new point to the line

This means we are **creating our own app class** called TouchDrawApp, and it **inherits** (takes features) from Kivy's built-in class **App**.

```
    def on_touch_move(self, touch):
        if 'line' in touch.ud:
            touch.ud['line'].points .pos

    def on_touch_up(self, tou
        # you could fade o move drawings here if you want
        pass

class TouchDrawApp(App):
    def build(self):
        Window.clearcolor = (0.09, 0.09
        return DrawingPad()

if __name__ == "__main__":
    TouchDrawApp().run()
```

What is App in Kivy?

App is a **base class** in Kivy that represents your whole **application**.

It controls how your program:

Starts

Builds the main interface (the window and widgets)

Runs the event loop (keeps your app responsive)

Closes when you exit

So basically, App = **the manager of your application's life cycle**.

Create a new application named TouchDrawApp that behaves like a Kivy app

```
    def on_touc          self, touch):
        if 'lin          touch.ud:
            tou          ['line'].points +=          s

    def on_touc          (self, touch):
        # you c          d fade or remo          awings here i
        pass

class TouchDrawApp(App):
    def build(self):
        Window.clearcolor = (0.09, 0.09,
        return DrawingPad()

if __name__ == "__main__":
    TouchDrawApp().run()
```

- When the app starts, Kivy automatically calls build().
- You set the background color.
- You return your main widget (DrawingPad) — that becomes the visible screen.

This does the following steps automatically:
Creates an instance of your app → TouchDrawApp().
Calls its build() method to create the interface.
Opens the Kivy window.
Starts listening for user interactions (touch, mouse, keyboard).
Keeps running until the user closes the window.

Thanks